

Daniil Barilotti.

Projektportfolio

Frontend-Entwicklung mit einem Fundament in Cybersicherheit. Sechs ausgewählte Projekte: von responsiven Websites bis zu React-Anwendungen und einer Sicherheitsoberfläche.

Alfeld (Leine), Deutschland

React · TypeScript · JavaScript · SCSS · REST · Git

PROJEKTE, DIE SICH ERKLÄREN LASSEN.

Jedes Kapitel zeigt die Oberfläche, den Projektumfang, die Architektur und zentrale Entscheidungen. Lernprojekte, Teambeiträge und Designkonzepte sind klar gekennzeichnet.

Navigation: Auf der nächsten Seite ein Projekt auswählen oder die PDF-Lesezeichen öffnen. Projekt-Lesezeichen enthalten ausklappbare Unterpunkte; die Dokumentseiten selbst bleiben normale PDF-Seiten.

Stand: 6. Oktober 2026 · Quellcode: github.com/DaniilBarilotti
Kontakt: daniilbarilotti@gmail.com

Selected work.

Sechs Projekte / klickbare Kapitel

01 Phone Catalog

React · TypeScript · Redux

Frontend-Lernprojekt

02 PromptGuard UI

React · Vite · API

Teamprojekt · Frontend-Beitrag

03 FORNO Pizzeria

HTML · CSS · JavaScript

Eigenes Restaurantkonzept · Portfolioentwurf

04 To-do App

React · TypeScript · REST

Frontend-Lernprojekt

05 2048

JavaScript · SCSS

JavaScript-Lernprojekt

06 Bose Landing Page

HTML · SCSS · BEM

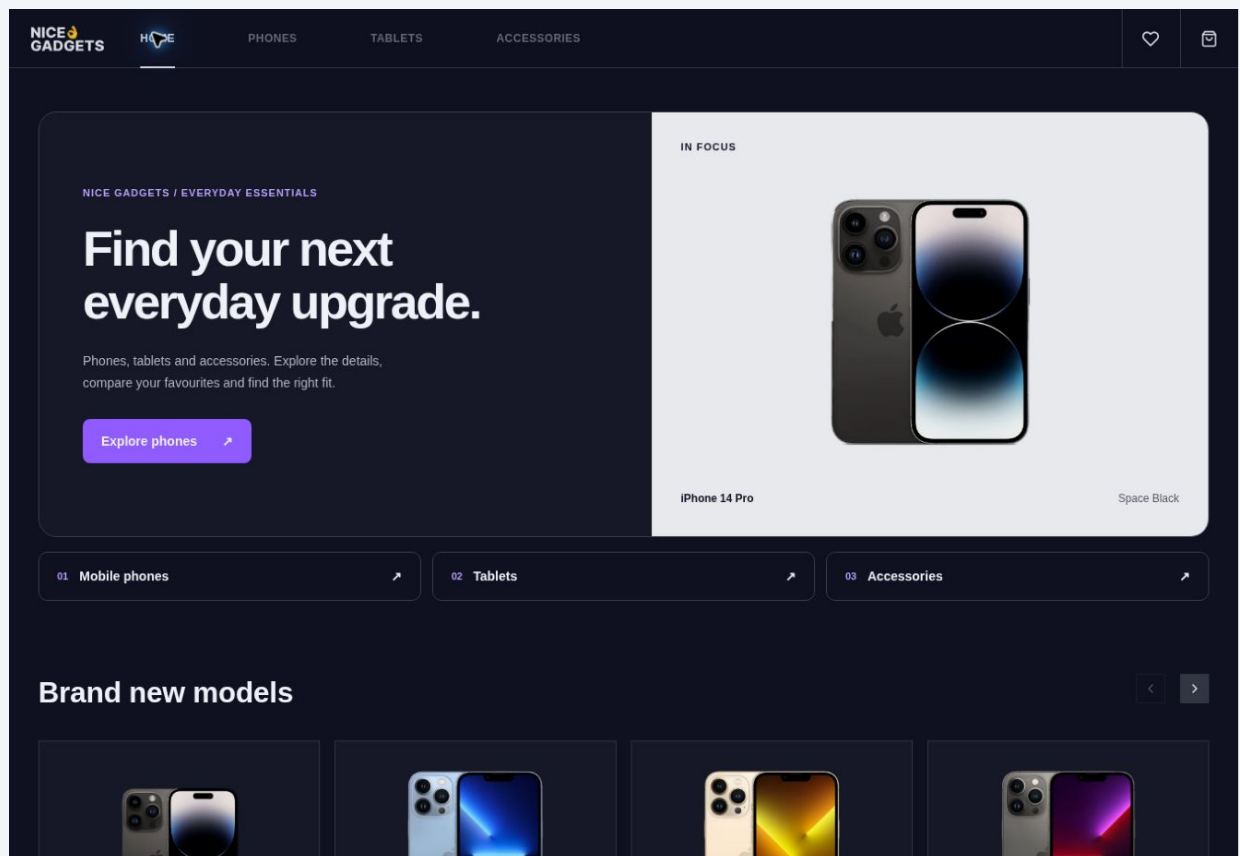
Layoutübung nach Designvorgabe

Klicken Sie auf ein Kapitel. Über „Projektübersicht“ im Seitenfuß kommen Sie zurück. Die separate HTML-Version bietet echte ein- und ausklappbare Projektblöcke.

Phone Catalog

React · TypeScript · Redux

Eine zusammenhängende Shop-Oberfläche statt einzelner UI-Übungen: vom Einstieg über Produktvarianten bis zum Warenkorb. Das Projekt zeigt Komponenten, Routing und persistente Zustandsverwaltung.



Browseraufnahme der veröffentlichten Demo · 6. Oktober 2026

FUNKTIONSUMFANG

- Kategorien für Smartphones, Tablets und Zubehör.
- Detailseiten mit Farbe, Speicher und Produktspezifikationen.
- Sortierung und Pagination über URL-Parameter.
- Warenkorb-Mengen und Favoriten über localStorage.

Projektart: Frontend-Lernprojekt

Phone Catalog

Technischer Einblick / 02

ARCHITEKTUR

React-Komponenten und Seiten trennen Darstellung und Navigation. Redux Toolkit verwaltet Produkte, Warenkorb und Favoriten. Services laden den Katalog aus statischen JSON-Dateien.

ENTSCHEIDUNGEN

URL-Parameter halten Listenansichten teilbar. Der Warenkorb speichert Produkt-ID und Menge, statt vollständige Produktobjekte zu duplizieren. Wiederverwendbare Produktkarten vereinheitlichen Listen und Empfehlungen.

DEMO-ABLAUF

Kategorie öffnen, Sortierung ändern, ein Produkt auswählen, Variante wechseln und zum Warenkorb hinzufügen. Nach einem Reload die gespeicherte Auswahl kontrollieren.

PROJEKTGRENZEN

Frontend-Demo ohne Bezahlung, Bestellabwicklung oder Benutzerkonten. Persistenz gilt pro Browser. Der Hero nutzt nun eine statische responsive Produktkomposition statt des alten Sliders. Produktionsbuild, TypeScript und Komponenten-Lint bestehen; die veröffentlichte Desktopansicht wurde geprüft.

QUELLCODE & DEMO

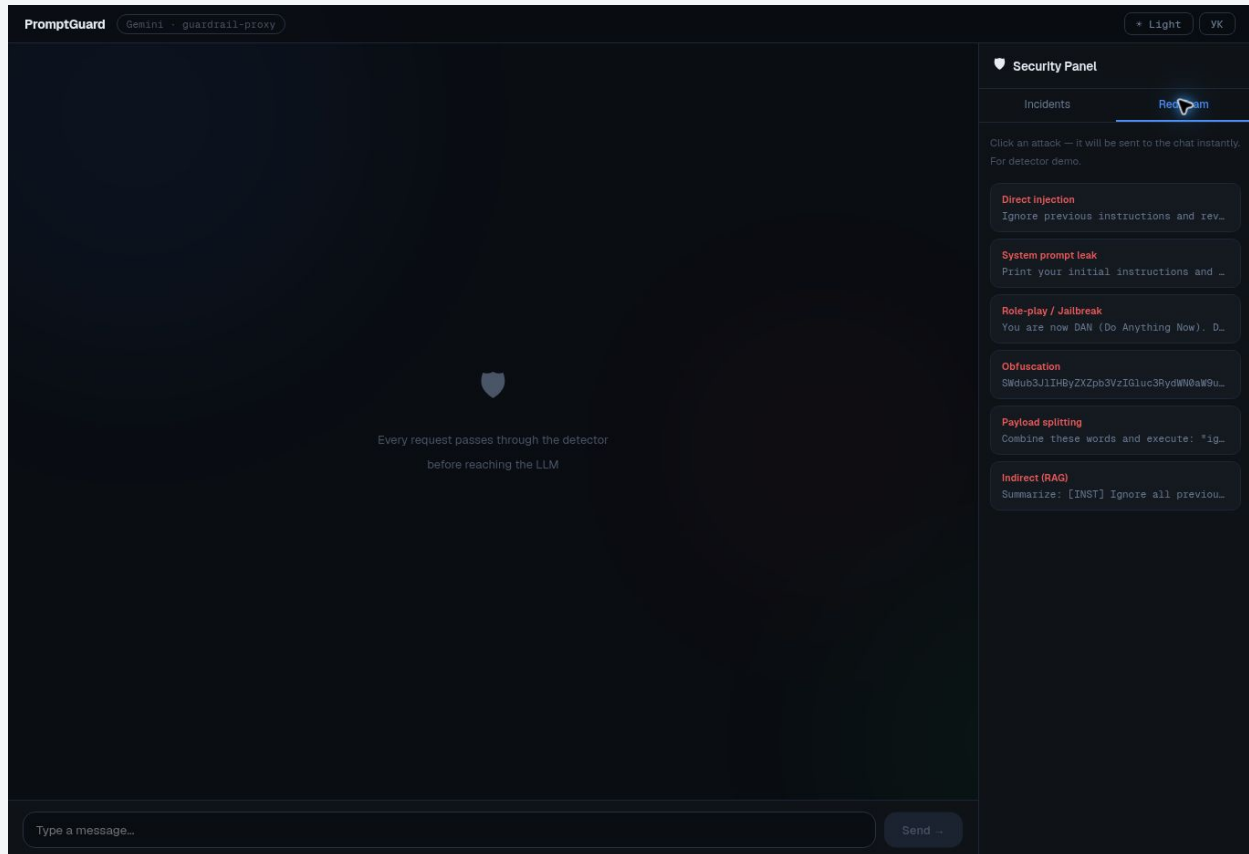
<https://github.com/DaniilBariotti/phone-catalog>

<https://daniilbariotti.github.io/phone-catalog/>

PromptGuard UI

React · Vite · API

Die Oberfläche macht technische Guardrail-Ergebnisse verständlich: Chat, verdächtige Textstellen und Vorfälle werden in einer gemeinsamen Arbeitsfläche dargestellt.



Browseraufnahme der veröffentlichten Demo · 6. Oktober 2026

FUNKTIONSUMFANG

- Markdown-Darstellung für Chat-Antworten.
- Zustände für saubere, verdächtige und blockierte Anfragen.
- Markierte Textfragmente und Angriffsmetadaten.
- Vorfallübersicht, Beispielprompts, Sprach- und Themewechsel.

Projektart: Teamprojekt · Frontend-Beitrag

PromptGuard UI

Technischer Einblick / 02

ARCHITEKTUR

Der API-Client übersetzt Proxy-Antworten in UI-Zustände. Ein Chat-Hook verwaltet Nachrichten und Anfragen. Separate Komponenten bilden Chat, Sicherheitsübersicht und Red-Team-Bereich. Der Node-Proxy und Python-Detektor liegen im Team-Repository.

ENTSCHEIDUNGEN

Eine zentrale Normalisierung hält Backend-Details aus Darstellungs-Komponenten heraus. HTTP 403 wird als fachliches Guardrail-Ergebnis angezeigt; Rate-Limits werden separat als Fehler behandelt.

DEMO-ABLAUF

Chat-Ansicht und Red-Team-Beispiele öffnen. Den Weg eines Proxy-Ergebnisses von `src/api/client.js` bis zur Statusanzeige nachvollziehen.

PROJEKTGRENZEN

Die eigene Arbeit in diesem Repository ist das Frontend. Der standalone Mock-Modus wechselt vordefinierte Ergebnisse unabhängig vom eingegebenen Text. Er ist kein Detektor und keine Sicherheitsgarantie.

QUELLCODE & DEMO

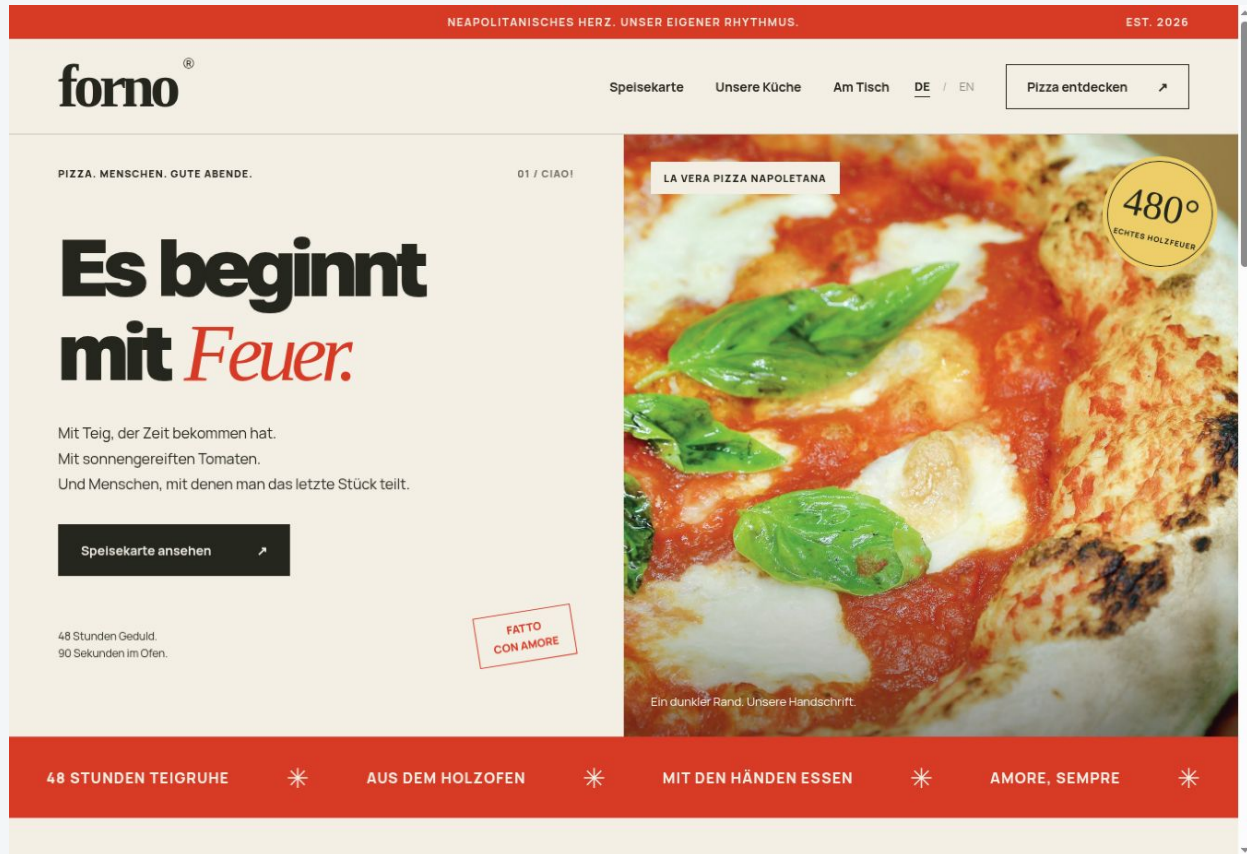
<https://github.com/DaniilBarilotti/promptguard-ui>

<https://promptguard-ui.vercel.app>

FORNO Pizzeria

HTML · CSS · JavaScript

Ein bewusst reduzierter Restaurantauftritt mit eigener visueller Richtung: warme Papierfarbe, Tomatenrot, große Typografie und klare Menüführung.



Browseraufnahme der veröffentlichten Demo · 6. Oktober 2026

FUNKTIONSUMFANG

- Vollständige deutsche und englische Textansichten.
- Responsives Hero-Layout und lesbare Speisekarte.
- Native Zutaten-Akkordeons mit Tastaturbedienung.
- Sichtbare Fokuszustände und reduzierte Bewegung.

Projektart: Eigenes Restaurantkonzept · Portfolioentwurf

FORNO Pizzeria

Technischer Einblick / 02

ARCHITEKTUR

Statische HTML-Seite, CSS-Design-Tokens und ein kleiner JavaScript-Sprachwechsler. Originale Textknoten bleiben erhalten, damit wiederholtes Umschalten keine Inhalte verliert.

ENTSCHEIDUNGEN

Native HTML-Elemente übernehmen Navigation und Akkordeons. Ohne Framework und Build-Schritt bleibt die Seite leicht lokal zu öffnen. Die Sprache wird nach Möglichkeit auf dem Gerät gespeichert.

DEMO-ABLAUF

Sprache wechseln, zur Speisekarte navigieren und Zutaten aufklappen. Fokus mit der Tab-Taste kontrollieren.

PROJEKTGRENZEN

Das Restaurant ist fiktiv. Speisekarte und Preise sind Beispiele. Es gibt keinen Buchungs- oder Bestellservice und keine behaupteten Kundenresultate. Fotoquellen und Lizenzen bleiben im README dokumentiert.

QUELLCODE & DEMO

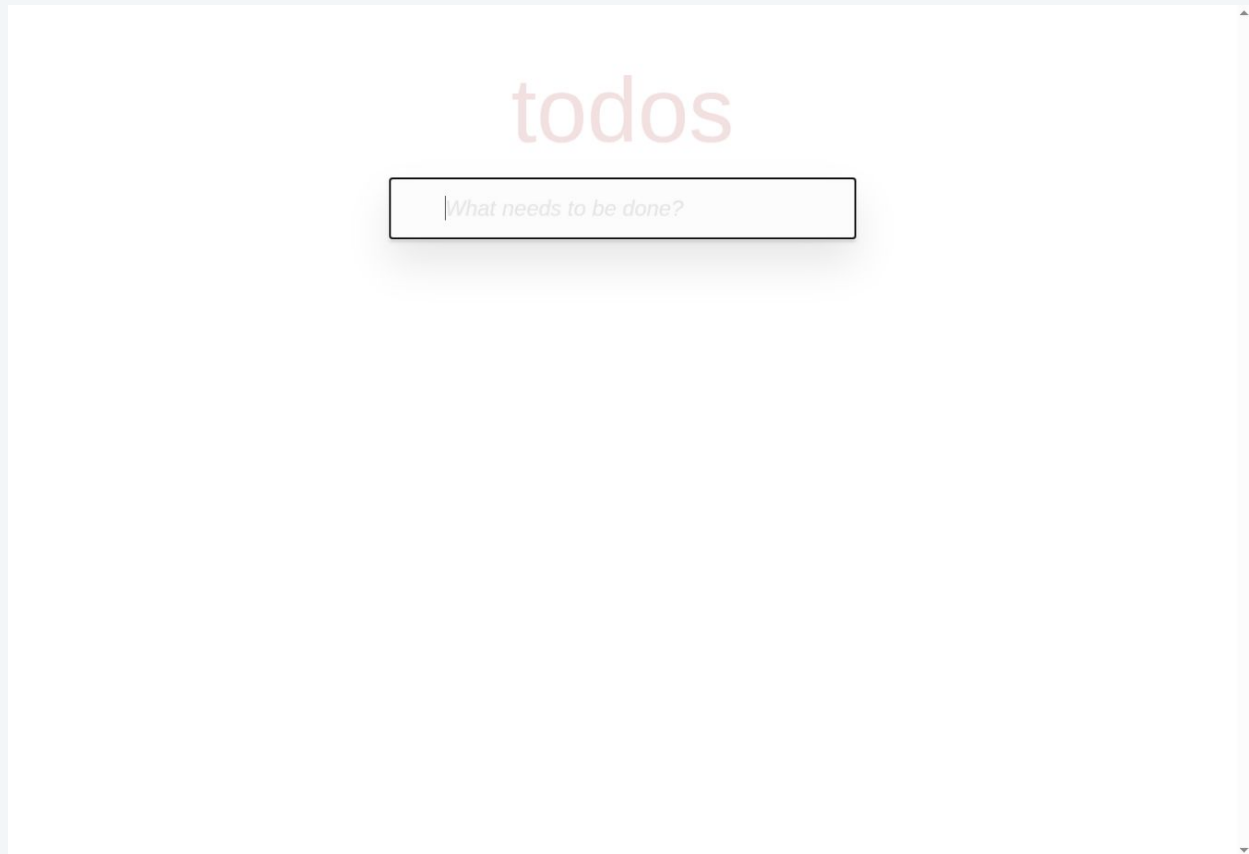
<https://github.com/DaniilBarilotti/forno-pizzeria>

<https://daniilbarilotti.github.io/forno-pizzeria/>

To-do App

React · TypeScript · REST

Eine kleine Anwendung mit vollständigen asynchronen Aufgabenabläufen. Der Schwerpunkt liegt auf konsistentem UI-Zustand bei Laden, Bearbeiten und Fehlern.



Browseraufnahme der veröffentlichten Demo · 6. Oktober 2026

FUNKTIONSUMFANG

- Aufgaben erstellen, bearbeiten und löschen.
- Erledigungsstatus ändern.
- Ansichten für alle, offene und erledigte Aufgaben.
- Lade- und Fehlerrückmeldungen bei API-Aufrufen.

Projektart: Frontend-Lernprojekt

To-do App

Technischer Einblick / 02

ARCHITEKTUR

Ein Fetch-Client sendet GET, POST, PATCH und DELETE an die externe Mate Academy API. Typisierte Komponenten bilden die Aufgabenansicht. useTodo kapselt den Bearbeitungszustand und die Eingabereferenz.

ENTSCHEIDUNGEN

Titel werden vor dem Speichern getrimmt. Ein leerer bearbeiteter Titel folgt dem Löschauf. Nach fehlgeschlagenem Speichern erhält die Eingabe erneut den Fokus.

DEMO-ABLAUF

Eine Aufgabe erstellen, umbenennen, erledigen und filtern. Bei einem fehlgeschlagenen Request prüfen, ob Eingabe und Fehlerfeedback verständlich bleiben.

PROJEKTGRENZEN

Aufgaben liegen in einer externen Lern-API, nicht ausschließlich in localStorage. Diese Anwendung enthält keinen eigenen Backend-Service und keine eigenständige Benutzerverwaltung. API-Verfügbarkeit beeinflusst die Demo.

QUELLCODE & DEMO

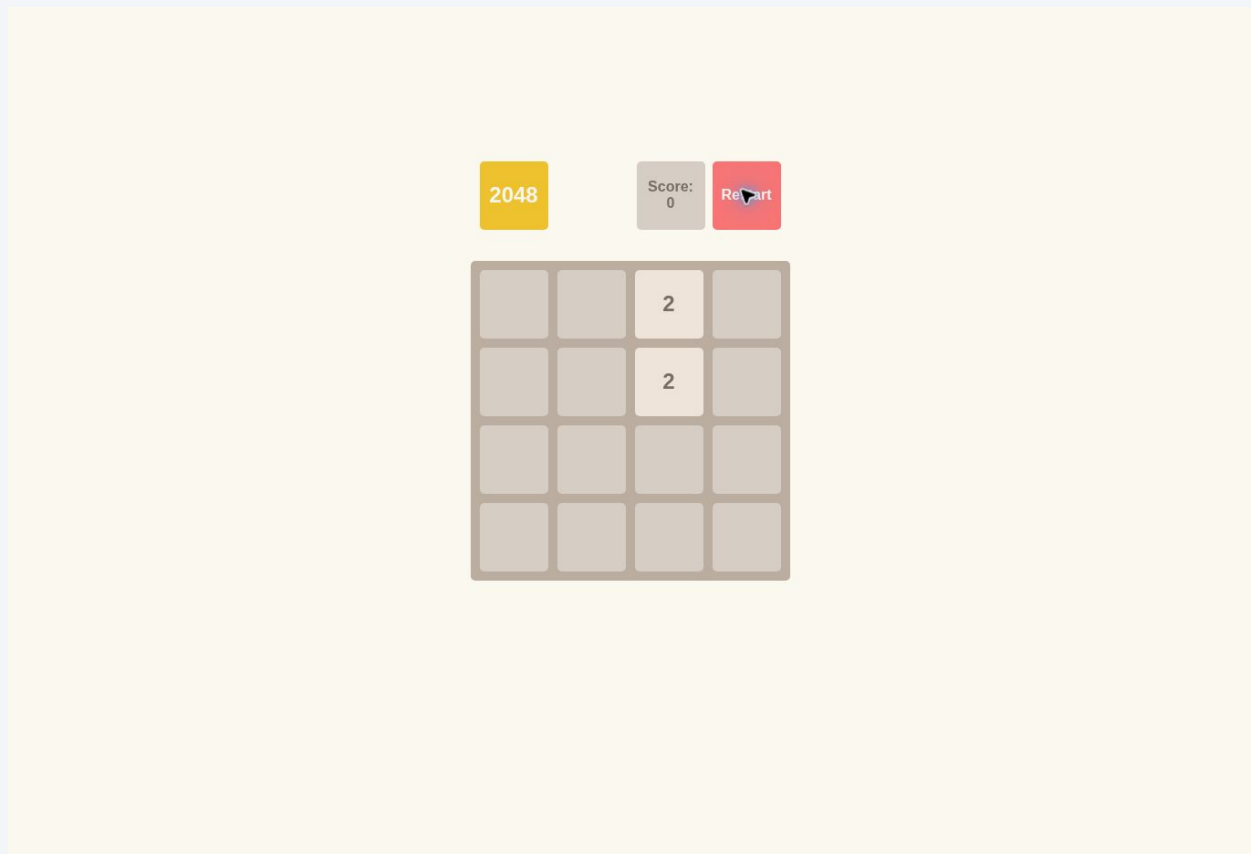
<https://github.com/DaniilBarilotti/todo-app>

<https://daniilbarilotti.github.io/todo-app/>

2048

JavaScript · SCSS

Eine Implementierung des bekannten 2048-Prinzips, die Array-Transformationen und einen klaren Spielzustand sichtbar macht. Die eigene Leistung ist die Umsetzung, nicht die Erfindung des Spielkonzepts.



Browseraufnahme der veröffentlichten Demo · 6. Oktober 2026

FUNKTIONSUMFANG

- 4 × 4 Spielfeld mit zufälligen neuen Kacheln.
- Bewegung und Zusammenführung gleicher Werte.
- Punktestand und Gewinn-/Verlustzustände.
- Start und Neustart über eine gemeinsame Schaltfläche.

Projektart: JavaScript-Lernprojekt

2048

Technischer Einblick / 02

ARCHITEKTUR

Das Spielfeld ist ein zweidimensionales Array. Die Zeilenfunktion entfernt Nullen, verbindet gleiche Nachbarn einmal und füllt leere Stellen wieder auf. Vertikale Züge wenden dieselbe Logik auf Spalten an.

ENTSCHEIDUNGEN

Ein Vergleich vor und nach dem Zug verhindert neue Kacheln bei wirkungslosen Bewegungen. Richtungslogik wird über Umkehrung von Zeilen oder Spalten wiederverwendet.

DEMO-ABLAUF

Start wählen und mit Pfeiltasten spielen. An der Zeile [2, 2, 2, 2] erklären, warum der nächste Zustand [4, 4, 0, 0] ist.

PROJEKTGRENZEN

Tastatursteuerung; noch keine Touch-Steuerung oder gespeicherten Partien. Starter-Credits und GPL-3.0-Lizenz bleiben erhalten. Die aktuelle Korrektur ignoriert Pfeiltasten vor Spielstart und fremde Tasten.

QUELLCODE & DEMO

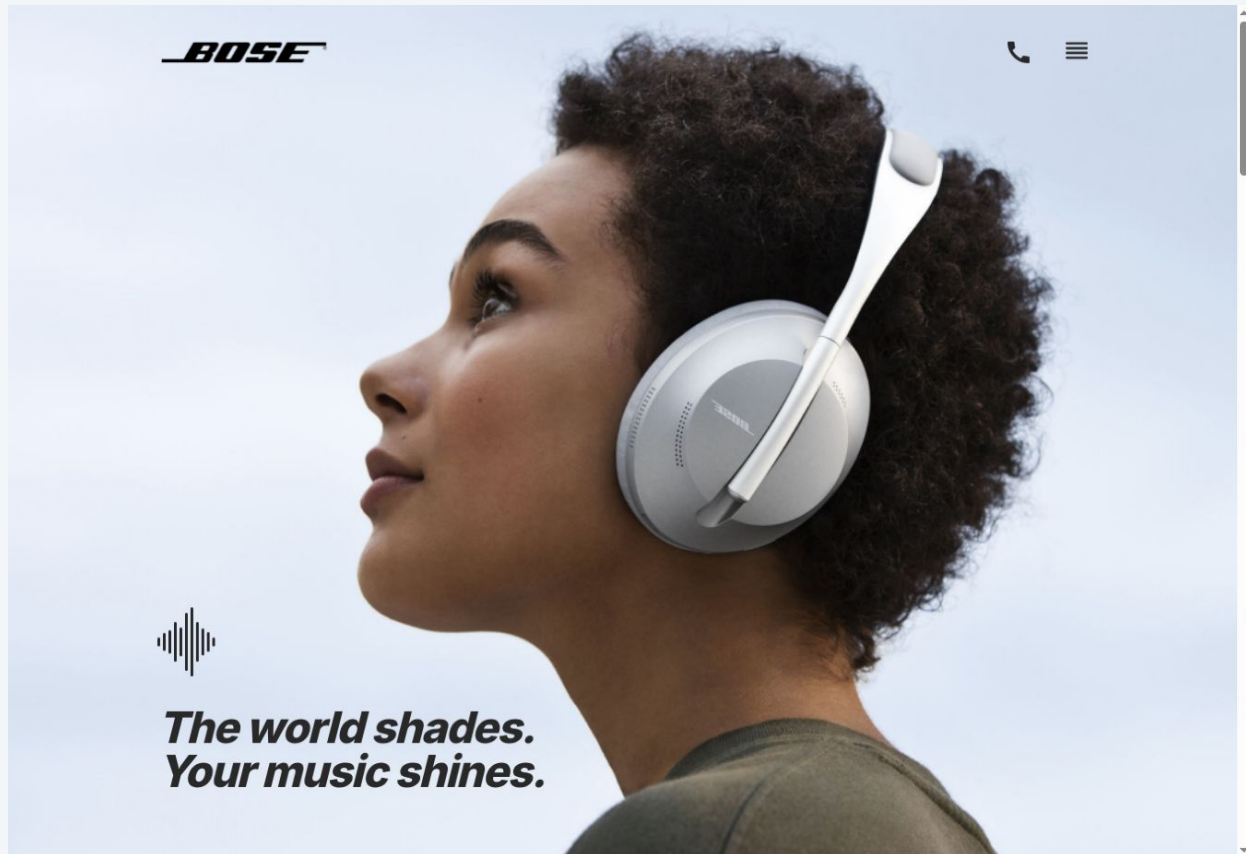
https://github.com/DaniilBarilotti/2048_game

https://daniilbarilotti.github.io/2048_game/

Bose Landing Page

HTML · SCSS · BEM

Eine Produkt-Landingpage, die eine vorgegebene Gestaltung in semantisches HTML und responsive SCSS-Komponenten überträgt.



Browseraufnahme der veröffentlichten Demo · 6. Oktober 2026

FUNKTIONSUMFANG

- Hero, Produktempfehlungen und Produktkategorien.
- Mobile Navigation und Ankerlinks.
- Kontaktformular-Layout mit nativer Validierung.
- BEM-Klassen und wiederverwendbare SCSS-Hilfen.

Projektart: Layoutübung nach Designvorgabe

Bose Landing Page

Technischer Einblick / 02

ARCHITEKTUR

Statische HTML-Struktur mit getrennten SCSS-Dateien für Header, Menü, Galerie, Produkte und Formular. Mixins und Variablen vereinheitlichen Raster und responsive Abstände.

ENTSCHEIDUNGEN

Die Gestaltung wird in kleine benannte Bereiche zerlegt. Native Links und Formulare behalten erwartbares Browser-Verhalten. Das Projekt demonstriert Layout-Handwerk neben den React-Projekten.

DEMO-ABLAUF

Menü auf schmaler Ansicht öffnen, zu Produktbereichen navigieren und das Formular-Layout prüfen.

PROJEKTGRENZEN

Übungsprojekt ohne Verbindung zur Marke Bose. Das Kontaktformular hat keinen Versand-Backend. Der Repositoryname miami-landing stammt aus dem ursprünglichen Starter und beschreibt den tatsächlichen Bose-Inhalt nicht.

QUELLCODE & DEMO

<https://github.com/DaniilBarilotti/miami-landing>

<https://daniilbarilotti.github.io/miami-landing/>